BTK AKADEMİ LMS • UYGULAMA ATÖLYESİ

Claude Code Agent Teams

Birden çok bağımsız Claude Code oturumuyla işi bölme, koordine etme ve sonucu doğrulama.

Amaç: kendi projenizde bir agent team kurup yönetebilmek

YOL HARİTASI

Bu sunumdan sonra ne yapabileceksiniz?

01

Karar ver

Bir görev için agent team'in uygun olup olmadığını ayırt et.

02

Kur ve böl

Özelliği aç, oturum başlat, görevleri ve dosya sahiplerini belirle.

03

Yönet ve doğrula

Ekip arkadaşlarını yönlendir, çıktıları birleştir, kabul ölçütlerini kontrol et.

İzleyeceğimiz örnek: LMS'de **kurs kataloğu arama ve filtreleme** işi (FR-26).

TEMEL MODEL

Bir ekip nasıl çalışır?

Takım lideri

Ana Claude Code oturumu

Görevleri tanımlar, işi dağıtır, gelişmeleri izler ve sonuçları birleştirir.

↔

Ortak görev listesi

Durum + bağımlılık

Bekliyor → sürüyor → tamamlandı. Uygun görevler sahiplenilebilir.

↔

Ekip arkadaşları

Bağımsız oturumlar

Her birinin kendi bağlamı vardır; birbirlerine mesaj gönderebilirler.

Subagent: dar işi yapıp sonucu çağırana döndürür.

Agent team: ekip arkadaşları ortak görevler ve doğrudan mesajlaşma ile koordine olur.

Kaynak: resmî doküman, “Compare with subagents”, “Architecture”

KARAR NOKTASI

Agent team ne zaman değer üretir?

Uygun işler

- Bağımsız modüller veya farklı katmanlar
- Paralel kod incelemesi ve araştırma
- Birden fazla hata hipotezinin sınanması
- Ekip arkadaşlarının birbirine bulgu aktarması gereken işler

Tek oturum daha uygun

- Adımlar kesin sırayla ilerliyorsa
- Herkes aynı dosyayı düzenleyecekse
- Görev çok küçük ve koordinasyon maliyeti yüksekse
- Yalnızca kısa, odaklı bir alt sonuç gerekiyorsa

> Başlangıç için resmî öneri: çoğu akışta **3–5 ekip arkadaşı**; maliyet ve koordinasyon gereksinimi ekip sayısıyla artar.

Kaynak: “When to use agent teams”, “Best practices”

ADIM 1 · ETKİNLEŞTİR

Deneysel özelliği aç ve oturum başlat

Seçenek A — kullanıcı ayarı: `~/.claude/settings.json` dosyasındaki env alanı:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Seçenek B — PowerShell oturumu:

```
$env:CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS = '1'
cd D:\btk-akademi\lms
claude
```

Kontrol listesi

1. Claude Code'un kurulu ve oturum açılmış olduğundan emin ol.
2. **Etkileşimli** terminal oturumu aç.
3. Takımı doğal dilde açıkça iste.
4. Windows için varsayılan **in-process** görünümü kullan.

Not: `claude -p` gibi etkileşimsiz kullanımda ekip arkadaşı başlatılmaz.

Kaynak: "Enable agent teams", "Choose a display mode"

ADIM 2 · BAĞLAMI OKU

Örnek: bu LMS deposunda ne var?

Mevcut yapı

- `web/`: React + TypeScript + Vite
- `web/src/features/courses/CoursesPage.tsx`: kayıtlı kurslar ve sabit örnek veri
- `CLAUDE.md`: bileşen, CSS token ve geliştirme kuralları
- `ui/DESIGN.md`: görsel tasarım referansı

Hedef iş

- `docs/gereksinimler.md` içindeki **FR-26**: katalogda kategori, seviye, etiket ve anahtar kelime filtresi
- Gereksinim belgesinde sunucu hedefi: Node.js + SQLite
- Bu depoda sunucu kodu henüz yok; API katmanı yeni bir çalışma alanı

Katalog örneği **öğretici senaryodur**; bu sunum kodu uygulamaz. Var olan “Kayıtlı Kurslarım” sayfasını yanlışlıkla katalog sanmayın.

Kaynak: depodaki CLAUDE.md ve docs/gereksinimler.md

ADIM 3 · İŞİ TANIMLA

Bitmiş sayılma koşullarını önce yaz

Kullanıcı davranışı

- Kurslar anahtar kelime, kategori, seviye ve etiketle filtrelenir.
- Birden çok filtre birlikte çalışır.
- Boş sonuç, yüklenme ve hata durumları anlaşılır.
- Klavye ile kullanım ve görünür odak sağlanır.

Teknik doğrulama

- API sözleşmesi giriş/çıkışı açıkça tanımlanır.
- Sunucu filtrelemeyi doğrular; istemci parametreleri güvenle üretir.
- Başarılı/boş/hatalı yanıtlar test edilir.
- `web/` için `npm run build` başarılıdır.

Kabul ölçütleri görev dağıtımı için ortak sözleşmedir.

ADIM 4 · ÇAKIŞMAYI ÖNLE

Her ekip arkadaşına ayrı alan ver

Rol	Sahip olduğu alan	Çıktı
frontend	`web/src/features/catalog/**`	Katalog arayüzü, durumlar, erişilebilirlik
backend	`server/src/catalog/**` (yeni alan)	API tasarımı/uygulaması ve filtreleme
reviewer	Salt okunur inceleme; rapor: `docs/catalog-review.md`	FR-26 kapsamı, sözleşme ve test boşlukları
lider	Paylaşılan yönlendirme dosyaları: `web/src/routes/index.tsx`, `web/src/layout/Sidebar/navItems.ts`	API sözleşmesi kararı, entegrasyon, son kontrol

Ekip arkadaşları aynı çalışma dizinindeki dosyaları görür. Aynı dosyayı iki kişiye eşzamanlı düzenletmeyin.

Kaynak: “Avoid file conflicts”, “Context and communication”

ADIM 5 · BAĞIMLILIKLARI KUR

Önce sözleşme, sonra paralel üretim

T0 · lider

API sözleşmesi

Filtre adları, yanıt alanları, hata biçimi.

→

T1 + T2

Paralel iş

Frontend ve backend ayrı dosyalarda ilerler.

→

T3 · reviewer

İnceleme

Gereksinim, sözleşme ve durumları denetler.

→

T4 · lider

Entegrasyon

Route ekler, build/test çalıştırır, farkları kapatır.

Görev durumu: pending → in progress → completed.

Bağımlılık: T1 ve T2, T0 netleşmeden başlayıp farklı sözleşmeler üretmemeli.

Görev araçları bulunmayan oturumlarda ekip mesajlarla koordine olur.

Kaynak: “Assign and claim tasks”, “Best practices”

KOPYALA VE UYARLAYABİLİRSİN · PROMPT 1

Takımı başlat

Bu depodaki LMS için FR-26 kurs kataloğu arama ve filtreleme özelliğini planla. Önce CLAUDE.md, docs/gereksinimler.md ve ui/DESIGN.md dosyalarını oku. Üç kişilik bir **agent team** kur: frontend, backend, reviewer.

İlk adımda GET /api/courses için sorgu parametreleri, yanıt alanları ve hata biçimini kısa bir sözleşme olarak öner; benim görebileceğim şekilde özetle. Ardından görev listesi ve bağımlılıkları kur. Frontend yalnız web/src/features/catalog/**, backend yalnız server/src/catalog/** dosyalarını düzenlesin. Reviewer kodu değiştirmeden bulguları docs/catalog-review.md dosyasına yazsın. Paylaşılan route/nav dosyalarını sen, lider olarak, entegrasyonda ele al.

FR-26; boş/yükleniyor/hata durumları; erişilebilirlik; test ve npm run build kabul ölçütleri olsun. Ekip arkadaşlarına eksiksiz görev bağlamı ver, bitmelerini bekle, sonra sonuçları doğrula ve kalan riskleri raporla.

Not: Claude bazen subagent açabilir; gerçekten agent team istediğini açıkça yinele.

Kaynak: “Start your first agent team”, “Give teammates enough context”

KOPYALA VE UYARLAYABİLİRSİN · PROMPT 2

Frontend arkadaşına net görev ver

frontend ekip arkadaşına şu görevi ver: FR-26 için kurs kataloğu arayüzünü web/src/features/catalog/** altında oluştur. Var olan CoursesPage kayıtlı kursları gösteriyor; onu kataloğa dönüştürme. CLAUDE.md ve ui/DESIGN.md kurallarına, ortak bileşenlere ve CSS token'larına uy.

Liderin kararlaştırdığı GET /api/courses sözleşmesini kullan. Anahtar kelime, kategori, seviye ve etiket filtreleri; birlikte filtreleme; yüklenme, boş sonuç ve hata durumları; klavye erişimi ve görünür odak ekle. web/src/routes/index.tsx ve web/src/layout/Sidebar/navItems.ts dosyalarını düzenleme. Tamamlayınca değiştirdiğin dosyaları, doğrulamaları ve entegrasyon için gerekenleri lidere bildir.

> İyi görev tanımı = **bağlam + kapsam + dosya sınırı + kabul ölçütü + rapor biçimi**.

Kaynak: “Give teammates enough context”, “Avoid file conflicts”

KOPYALA VE UYARLAYABİLİRSİN · PROMPT 3

Backend arkadaşına ayrı alan ver

backend ekip arkadaşına şu görevi ver: docs/gereksinimler.md içindeki FR-26 ve teknoloji varsayımlarını oku. Henüz sunucu kodu olmayan bu depoda, liderin kararlaştırdığı GET /api/courses sözleşmesine uyan katalog modülünü server/src/catalog/** altında geliştir. Node.js ve SQLite hedefini izle; destekleyici paket/altyapı gerekiyorsa önce lidere bildir.

Anahtar kelime, kategori, seviye, etiket parametrelerini doğrula; birlikte filtrele; boş sonuç ve hata yanıtlarını sözleşmeye uygun üret. Kapsam dışındaki web/** dosyalarına dokunma. Değişiklikleri, çalıştırma adımlarını, test sonuçlarını ve açık bağımlılıkları lidere bildir.

Sunucu iskeleti yoksa lider önce ortak altyapı kararı vermeli. Bu, örnek akışın bilinçli entegrasyon kapısıdır.

KOPYALA VE UYARLAYABİLİRSİN · PROMPT 4

Bağımsız inceleme yaptır

reviewer ekip arkadaşına şu görevi ver: FR-26 ve kabul ölçütlerini temel alarak frontend ile backend çıktısını incele. Kod dosyalarını değiştirme. Özellikle filtrelerin birlikte çalışmasını, istemci–API sözleşmesini, boş/yükleniyor/hata durumlarını, klavye erişimini ve doğrulama kanıtlarını kontrol et.

Bulguları önem derecesi, ilgili dosya/satır, beklenen davranış ve önerilen düzeltmeyle docs/catalog-review.md içine yaz. Yanlış alarm olabilecek noktaları ayrıca belirt. Frontend ve backend arkadaşlarına sözleşme uyuşmazlığını doğrudan mesajla bildir; özeti lidere gönder.

Bağımsız inceleme, paralel çalışmanın kalite kapısıdır.

ADIM 6 · İZLE VE YÖNLENDİR

Takımı çalışırken yönet

Terminalde

- Varsayılan **in-process** panelinde ↑/↓ ile ekip arkadaşını seç.
- **Enter** ile konuşmasını aç ve doğrudan yaz.
- **Ctrl+T** ile görev listesini göster/gizle.
- İş bitince çıktı ve görev durumunu kontrol et.

Kısa yönlendirme prompt'ları

```
frontend'e sor: FR-26 kabul ölçütlerinden
hangileri tamamlandı, hangileri bekliyor?

backend'e bildir: kategori ve seviye
parametreleri API sözleşmesindeki adlarıyla
birebir eşleşsin.

Ekip arkadaşlarının görevleri bitmeden
entegrasyona geçme.
```

> Ekip arkadaşları liderin konuşma geçmişini almaz; bağlamı mesajda ver. İzin istekleri lider oturumunda görünür.

Kaynak: "Talk to teammates directly", "Context and communication"

ADIM 7 · DOĞRULA

“Tamamlandı” demeden önce kanıt iste

1

Dosyaları karşılaştır

İki ekip arkadaşı aynı dosyayı düzenledi mi? API alanları ve filtre adları eşleşiyor mu?

2

Çalıştır

`web/` içinde `npm run build`; sunucu için gerçek test/başlatma komutları.

3

Kabul ölçütünü dene

Birlikte filtreleme, boş sonuç, hata, klavye erişimi ve responsive görünüm.

Lider prompt’u: **“Tüm görevleri ve reviewer bulgularını özetle; kritik açıkları gider, sonra kabul ölçütleri için kanıt tablosu ver.”**

Kaynak: “Wait for teammates to finish”, “Monitor and steer”

SIK KARŞILAŞILAN DURUMLAR

Takım beklediğiniz gibi davranmazsa

Belirti	İlk kontrol / eylem
Takım görünmüyor	Deneysel ayarı, etkileşimli oturumu ve açık “agent team” isteğini kontrol et.
Görev takılı kaldı	Çıktıyı incele; iş bittiyse görev durumunu düzeltmesini lidere söyle.
Ekip üyesi erken durdu	Transkripti oku; somut devam talimatı ver veya yerine yeni üye iste.
Oturum yeniden açıldı	`/resume`, in-process ekip üyelerini geri getirmez; yenilerini oluştur.

Diğer sınırlar: oturum başına tek takım; iç içe takım yok; kapatma gecikebilir; çok üye daha çok token tüketir.

Kaynak: “Troubleshooting”, “Limitations”, “Token usage”

UYGULAMA ÖDEVİ

Şimdi kendi projenizde deneyin

Doldurulabilir şablon

```
[PROJE] içinde [GEREKSİNİM] işini ele al.
Önce [BAĞLAM DOSYALARI] dosyalarını oku.
[ROL 1], [ROL 2], [REVIEWER] adlı 3 kişilik
bir agent team kur.

Önce [ORTAK SÖZLEŞME] üzerinde uzlaş.
Her role ayrı dosya alanı ver: [YOLLAR].
Görev bağımlılıklarını ve kabul ölçütlerini yaz.
Ekip bitince [TEST KOMUTLARI] çalıştır,
bulguları gider ve kanıtları raporla.
```

Son kontrol

1. İş parçaları birbirinden bağımsız mı?
2. Dosya sahipliği açık mı?
3. Her ekip arkadaşı yeterli bağlam alıyor mu?
4. Bağımlılıklar tanımlı mı?
5. Sonuç test ve kabul ölçütüyle doğrulandı mı?

Kopyalanabilir tam prompt'lar: **ornek-promptlar.md**